GUÍA DEL ALUMNO

Cuentas reales. Datos en la nube.

TaskFlow con Firebase: **VueFire, autenticación, Firestore en tiempo real, reglas de seguridad y modo sin conexión.**

Integración con Firebase
Vue 3.5 · Firebase 12 · VueFire 3

Oscar Alvarado
Frontend Tech Lead

Universidad San Martín de Porres
Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

TaskFlow, ahora con Firebase

Seguimos con **TaskFlow**, la app de las cuatro sesiones del módulo. Ya tiene **estado** (Pinia) y **páginas** (Vue Router). Hoy sus usuarios y sus tareas dejan de ser de mentira: viven en **Firebase**. Cada tema es una página corta de teoría y, enseguida, el código para escribirlo.

Los cuatro bloques de hoy

R1
SDK modular de Firebase y VueFire

`¿Cómo conectamos TaskFlow a un backend real?initializeApp · VueFire`

R2
Firebase Auth

`¿Cómo sabe TaskFlow quién es el usuario?useCurrentUser · getCurrentUser`

R3
Firestore en tiempo real

`¿Cómo guardamos y mostramos las tareas sin recargar?useCollection · addDoc`

R4
Reglas de seguridad y offline

`¿Qué impide que otro lea mis tareas y qué pasa sin red?firestore.rules · localCache`

TaskFlow a lo largo del módulo

SESIÓN	QUÉ SE AGREGA A TASKFLOW
1 · Estado	Stores de tareas, tema y sesión, y Tailwind.
2 · Vue Router	Páginas: proyectos, detalle con pestañas, login, guardias, carga diferida y transiciones.
3 · Firebase (hoy)	Cuentas reales, tareas en Firestore en tiempo real, reglas de seguridad y modo sin conexión.
4 · Acceso	Permisos reales por rol (admin / miembro), también en el servidor.

Antes de la clase: tu propio proyecto de Firebase

1. En **console.firebase.google.com**, crea un proyecto (sin Google Analytics; el plan gratuito basta, **no pide tarjeta**).
2. En *Configuración del proyecto* → *Tus apps*, registra una **app web** (`</>`) y copia los cuatro valores de `firebaseConfig`: `apiKey`, `authDomain`, `projectId` y `appId`.
3. En *Authentication* → *Método de acceso*, activa **Correo electrónico/contraseña** y **Google**.
4. En *Firestore Database*, crea la base en **modo de prueba** (en el último bloque la protegemos).

Cómo seguir la clase

- Parte de `src/checkpoints/inicio/` del proyecto de ejemplos: es TaskFlow al terminar la sesión 2.
- Cada bloque termina en un **checkpoint** (`r1` … `r4`): una carpeta `src/` completa. Si te pierdes, pégala dentro de tu `src/` y sigue.
- Node 22.18 o superior y VS Code con **Vue (Official)**. Tus claves van en `.env.local`, que **no se sube a git**.

Conectar TaskFlow a Firebase

CONCEPTO VueFire

TaskFlow guarda sus datos en un servidor simulado que olvida todo al recargar. ¿Cómo lo conectamos a un backend real?

QUÉ ES

El SDK modular de Firebase (importas solo lo que usas) y `VueFire`, el plugin que lo vuelve reactivo.

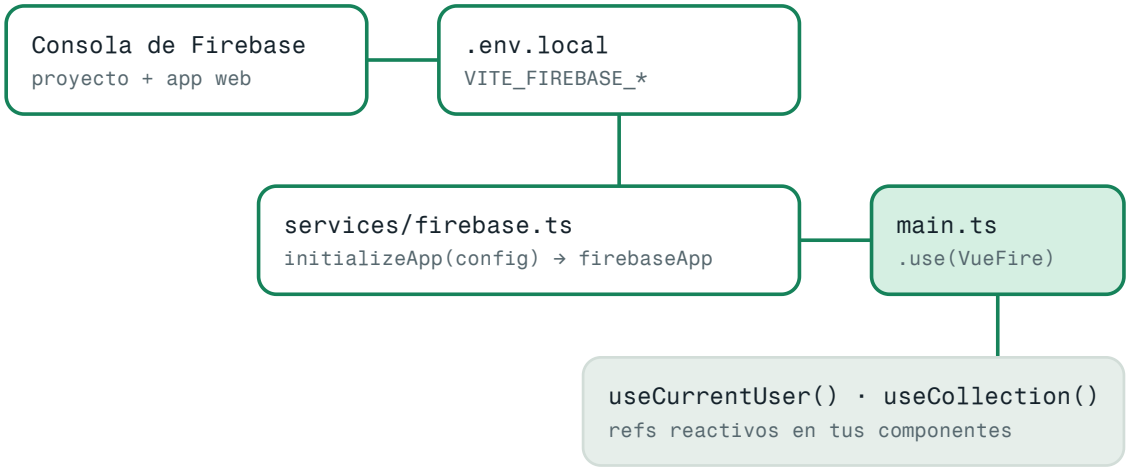
PARA QUÉ SE USA

Tener base de datos y cuentas sin montar un servidor, con datos que Vue sigue solo.

Paso 1 de 4



Paso 4 de 4



Cómo funciona, paso a paso

1. Crear el proyecto
2. Variables de entorno
3. initializeApp
4. Plugin VueFire

REGLA

Las claves del cliente **no son secretas**: la seguridad está en las reglas de Firestore, no en esconder el `.env`.

LO QUE AGREGAMOS A TASKFLOW

- proyecto de Firebase propio
- `services/firebase.ts`
- VueFire instalado

Bloque R1 de 4 · SDK modular y VueFire

1. **Terminal** — `npm install firebase@12 vuefire`. Se usa la versión 12 porque VueFire 3.2 declara compatibilidad hasta esa.
2. **.env.local** — ⚡ copia `.env.local.example` y pega los cuatro valores de tu app web. Vite solo expone las variables que empiezan por `VITE_`. Reinicia `npm run dev`.
3. **services/firebase.ts** — `initializeApp` con tu configuración y `getFirestore`. Exporta `firebaseApp` y `db`.
4. **main.ts** — `app.use(VueFire, { firebaseApp, modules: [VueFireAuth()] })`, después de Pinia y antes del router.

OJO

Las claves de `firebaseConfig` **no son secretas**: cualquiera las ve en el navegador. Lo que protege tus datos son las **reglas de Firestore** (bloque R4). Si falta el `.env.local`, la página queda en blanco y la consola dice `auth/invalid-api-key`.

src/services/firebase.ts

```
import { initializeApp } from 'firebase/app'
import { getFirestore } from 'firebase/firestore'

// Estas claves no son secretas: solo identifican tu proyecto. La seguridad la dan las
   reglas.
export const firebaseApp = initializeApp({
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
})

export const db = getFirestore(firebaseApp)
```

src/main.ts

```ts
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import { VueFire, VueFireAuth } from 'vuefire'
import App from './App.vue'
import { router } from './router'
import { firebaseApp } from './services/firebase'
import { precargar, enReposo } from './router/precarga'
import './assets/main.css'

const pinia = createPinia()

createApp(App)
  .use(pinia)
  .use(VueFire, { firebaseApp, modules: [VueFireAuth()] })
  .use(router)
  .mount('#app')

// Cuando el navegador no tiene nada que hacer, bajamos la vista que probablemente
   sigue: la lista
router.isReady().then(() => enReposo(() => precargar({ name: 'proyectos' })))
```

Quién es el usuario

CONCEPTO useCurrentUser

Firebase ya está conectado. ¿Cómo sabe TaskFlow quién es el usuario?

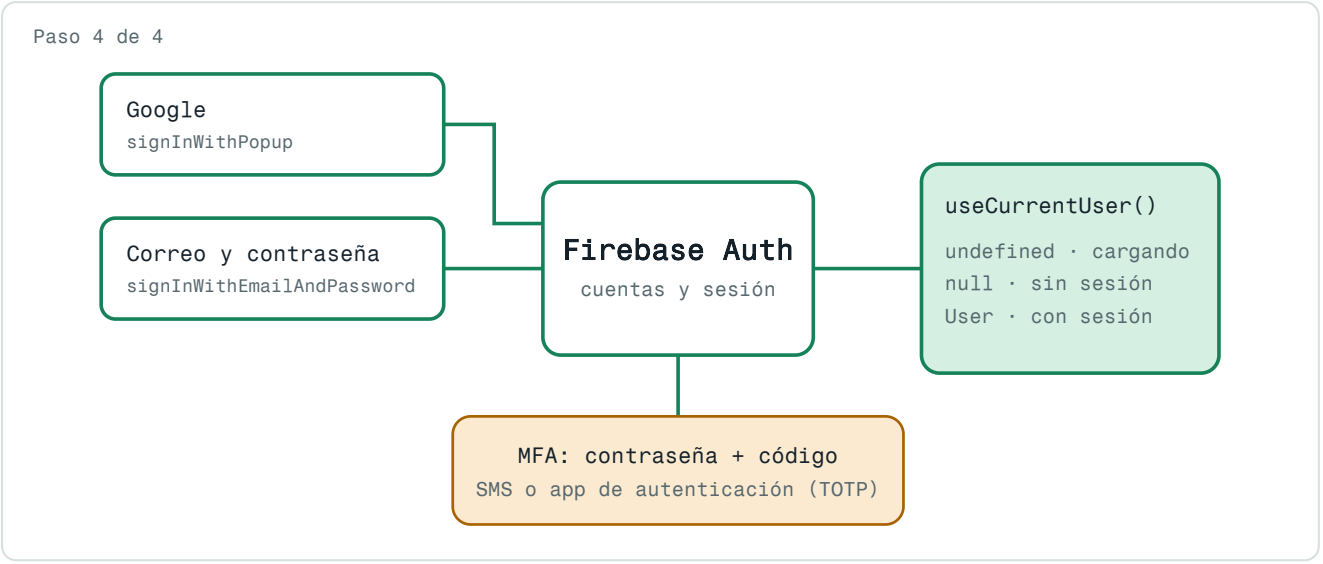
QUÉ ES

Firebase Auth identifica al usuario (correo, Google u otro proveedor) y recuerda su sesión; VueFire la expone como un ref.

PARA QUÉ SE USA

Reemplazar a Ana y Luis por cuentas reales, con un segundo factor opcional (MFA).





Cómo funciona, paso a paso

1. Correo y contraseña
2. Proveedor externo
3. Sesión reactiva
4. Segundo factor (MFA)

REGLA

El usuario llega de forma **asíncrona**: la guardia espera a `getCurrentUser()` antes de decidir.

LO QUE AGREGAMOS A TASKFLOW

- login con correo y con Google
- sesión que sobrevive a F5
- guardia que espera al usuario

Bloque R2 de 4 · Firebase Auth

1. **stores/sesion.ts** — el store conserva `usuario`, `autenticado` y `esAdmin`; por dentro `useCurrentUser()` reemplaza a los usuarios de demostración. Añade `registrar`, `entrar`, `entrarConGoogle` y `salir`.
2. **router/index.ts** — la guardia pasa a ser `async` y espera `await getCurrentUser()` antes de decidir.
3. **views/LoginView.vue** — ⚡ correo, contraseña, «crear cuenta» y el botón de Google. Los códigos `auth/…` se traducen a una frase.
4. **Navegador y consola** — crea una cuenta, recarga en una ruta protegida (sigues dentro) y mira *Authentication → Usuarios*.

MFA: SOLO LA IDEA

La **autenticación multifactor** pide un segundo factor (un código por SMS o de una app) además de la contraseña. Se activa en el proyecto de Firebase; hoy solo la estudiamos.

OJO

Sin `await getCurrentUser()`, un F5 en una ruta protegida te manda al login **aunque tengas sesión**: la guardia decide antes de que Firebase responda. Todas las cuentas son `admin` por ahora; en la sesión 4 los roles vendrán del servidor.

src/stores/sesion.ts

```
import { computed } from 'vue'
import { defineStore } from 'pinia'
import {
  GoogleAuthProvider,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signInWithPopup,
  signOut,
} from 'firebase/auth'
import { useCurrentUser, useFirebaseAuth } from 'vuefire'
import type { Usuario } from '../types'

export const useSesionStore = defineStore('sesion', () => {
  const auth = useFirebaseAuth()!
  const actual = useCurrentUser() // undefined mientras carga · null sin sesión

  // Firebase no trae roles: en la sesión 4 vendrán de los claims. Por ahora todos son
   admin.
```

```
  const usuario = computed<Usuario | null>(() =>
    actual.value
      ? {
          id: actual.value.uid,
          nombre: actual.value.displayName ?? actual.value.email ?? 'Tú',
          rol: 'admin',
        }
      : null,
  )
  const autenticado = computed(() => usuario.value !== null)
  const esAdmin = computed(() => usuario.value?.rol === 'admin')

  const registrar = (correo: string, clave: string) =>
    createUserWithEmailAndPassword(auth, correo, clave)
  const entrar = (correo: string, clave: string) => signInWithEmailAndPassword(auth, correo,
   clave)
  const entrarConGoogle = () => signInWithPopup(auth, new GoogleAuthProvider())
  const salir = () => signOut(auth)

  return { usuario, autenticado, esAdmin, registrar, entrar, entrarConGoogle, salir }
})
```

src/router/index.ts

```
import { createRouter, createWebHistory, type RouteLocationNormalized } from 'vue-router'
import { instalarTransiciones } from './transiciones'
import { getCurrentUser } from 'vuefire'
import { useSesionStore } from '../stores/sesion'

// Los parámetros de la URL siempre llegan como texto: se convierten aquí, una sola vez
const idDe = (route: RouteLocationNormalized) => Number((route.params as { id: string }).id)

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'inicio', redirect: { name: 'proyectos' } },
    {
      path: '/proyectos',
      name: 'proyectos',
      component: () => import('../views/ProyectosView.vue'),
      meta: { requiereAuth: true },
    },
    { path: '/login', name: 'login', component: () => import('../views/LoginView.vue') },
    {
      // Ruta padre: pinta el encabezado del proyecto y, dentro, un <RouterView> para el
   hijo
      path: '/proyectos/:id',
      component: () => import('../views/ProyectoView.vue'),
      meta: { requiereAuth: true },
      props: (route) => ({ id: idDe(route) }),
      children: [
        {
          path: '',
          name: 'proyecto-tareas',
          component: () => import('../views/TareasView.vue'),
          props: (route) => ({ proyectoId: idDe(route) }),
        },
        {
          path: 'ajustes',
          name: 'proyecto-ajustes',
          component: () => import('../views/ProyectoAjustes.vue'),
          meta: { soloAdmin: true },
```

```
        },
      ],
    },
    // Siempre al final: captura cualquier URL que no coincidió con las anteriores
    {
      path: '/:pathMatch(.*)*',
      name: 'no-encontrada',
      component: () => import('../views/NoEncontrada.vue'),
    },
  ],
})

// Guardia global: corre antes de CADA navegación. Devolver una ruta la redirige; no
   devolver nada la deja pasar.
router.beforeEach(async (to) => {
  // Firebase tarda un instante en saber quién es el usuario: la guardia espera esa
   respuesta
  await getCurrentUser()
  const sesion = useSesionStore()

  if (to.meta.requiereAuth && !sesion.autenticado) {
    // Guardamos a dónde quería ir para volver después de entrar
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  if (to.meta.soloAdmin && !sesion.esAdmin) {
    return { name: 'proyectos' }
  }
  if (to.name === 'login' && sesion.autenticado) {
    return { name: 'proyectos' }
  }
})

instalarTransiciones(router)
```

src/views/LoginView.vue

```
<script setup lang="ts">
import { ref } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { FirebaseError } from 'firebase/app'
import { useSesionStore } from '../stores/sesion'

const sesion = useSesionStore()
const route = useRoute()
const router = useRouter()

// `redirect` viene de la guardia. Solo aceptamos rutas internas ("/algo"), nunca "//otro-
   sitio"
const redirect = route.query.redirect
const destino =
  typeof redirect === 'string' && redirect.startsWith('/') && !redirect.startsWith('//')
    ? redirect
    : '/proyectos'

const correo = ref('')
const clave = ref('')
const creando = ref(false)
const error = ref<string | null>(null)

// Firebase responde con códigos ("auth/…"): al usuario le mostramos una frase
const MENSAJES: Record<string, string> = {
  'auth/invalid-credential': 'Correo o contraseña incorrectos.',
```

```
  'auth/user-not-found': 'Correo o contraseña incorrectos.',
  'auth/wrong-password': 'Correo o contraseña incorrectos.',
  'auth/email-already-in-use': 'Ya existe una cuenta con ese correo.',
  'auth/weak-password': 'La contraseña debe tener al menos 6 caracteres.',
  'auth/invalid-email': 'El correo no es válido.',
  'auth/popup-closed-by-user': 'Cerraste la ventana de Google antes de terminar.',
  'auth/network-request-failed': 'No hay conexión con Firebase.',
}

async function intentar(accion: () => Promise<unknown>) {
  error.value = null
  try {
    await accion()
    router.push(destino)
  } catch (e) {
    error.value =
      e instanceof FirebaseError
        ? (MENSAJES[e.code] ?? `Error de Firebase (${e.code})`)
        : 'Error desconocido'
  }
}

const enviar = () =>
  intentar(() =>
    creando.value
      ? sesion.registrar(correo.value, clave.value)
      : sesion.entrar(correo.value, clave.value),
  )
</script>

<template>
  <section class="mx-auto flex w-full max-w-sm flex-col gap-4 py-8">
    <h2 class="text-2xl font-bold">{{ creando ? 'Crear cuenta' : 'Entrar a TaskFlow' }}</h2>
    <p v-if="route.query.redirect" class="text-sm text-slate-500">
      Para ver <code>{{ route.query.redirect }}</code> primero inicia sesión.
    </p>

    <form class="flex flex-col gap-3" @submit.prevent="enviar">
      <input
        v-model="correo"
        type="email"
        required
        autocomplete="email"
        placeholder="correo@ejemplo.com"
        class="rounded border border-slate-300 bg-white px-3 py-2 focus:ring-2 focus:ring-
   sky-500 focus:outline-none dark:border-slate-600 dark:bg-slate-800"
      />
      <input
        v-model="clave"
        type="password"
        required
        :autocomplete="creando ? 'new-password' : 'current-password'"
        placeholder="Contraseña"
        class="rounded border border-slate-300 bg-white px-3 py-2 focus:ring-2 focus:ring-
   sky-500 focus:outline-none dark:border-slate-600 dark:bg-slate-800"
      />
      <button type="submit" class="rounded bg-sky-600 px-4 py-2 text-white hover:bg-sky-
   700">
        {{ creando ? 'Crear cuenta' : 'Entrar' }}
      </button>
    </form>

    <button
```

```
        type="button"
        class="rounded-lg border border-slate-200 bg-white px-4 py-2 shadow-sm hover:shadow-md
    dark:border-slate-700 dark:bg-slate-800"
        @click="intentar(sesion.entrarConGoogle)"
      >
        Continuar con Google
      </button>

      <p
        v-if="error"
        class="rounded border border-red-300 bg-red-50 px-3 py-2 text-sm text-red-800
    dark:border-red-800 dark:bg-red-950 dark:text-red-200"
        role="alert"
      >
        {{ error }}
      </p>

      <button
        type="button"
        class="text-sm text-sky-700 underline dark:text-sky-300"
        @click="creando = !creando"
      >
        {{ creando ? 'Ya tengo cuenta' : 'No tengo cuenta: crear una' }}
      </button>
    </section>
  </template>
```

Tareas en la nube, siempre al día

CONCEPTO useCollection

Ya hay usuarios reales. ¿Cómo guardamos y mostramos sus tareas sin recargar?

QUÉ ES

Firestore es la base de datos en la nube de Firebase: NoSQL, sin tablas ni SQL, guarda **documentos** (objetos) dentro de **colecciones**. Con `useCollection`, una consulta queda enlazada a un ref y cada cambio actualiza la lista.

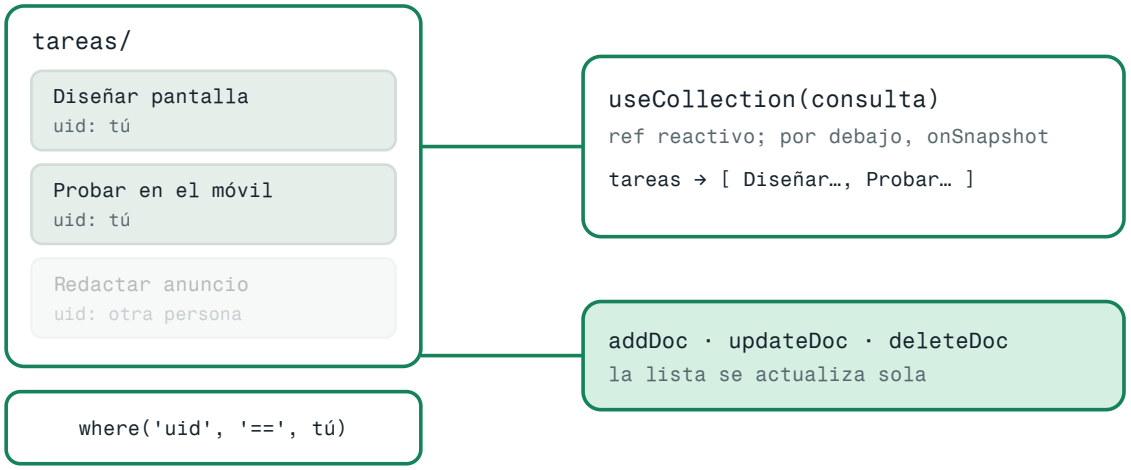
PARA QUÉ SE USA

Listas siempre al día, compartidas entre pestañas y dispositivos.

Paso 1 de 4



Paso 4 de 4



Cómo funciona, paso a paso

1. Colección y documentos
2. Consulta con where
3. useCollection
4. Escribir: add y update

REGLA

Una consulta solo **pide** datos; quien decide qué se puede leer son las reglas.

LO QUE AGREGAMOS A TASKFLOW

- tareas guardadas en Firestore
- lista que se actualiza sola

Bloque R3 de 4 · Firestore en tiempo real

1. **stores/tareas.ts** — `useCollection` recibe una función que devuelve `query(collection(db, 'tareas'), where('uid', '==', …))` o `null` si no hay usuario. Cuando el usuario cambia, VueFire abre otra consulta.
2. **stores/tareas.ts** — las acciones **escriben** con `addDoc`, `updateDoc` y `deleteDoc`; la lista cambia sola cuando la consulta avisa.
3. **types.ts, vistas, App.vue** — ⚡ el `id` de la tarea pasa a ser texto y lleva el `uid` de su dueño. Borra `services/tareasApi.ts`.
4. **Navegador y consola** — agrega una tarea y mírala en *Firestore → Datos*. Abre una segunda pestaña: lo que cambies en una aparece solo en la otra.

OJO

Una consulta solo **pide** datos; quien decide qué se puede leer son las reglas. Por eso la consulta filtra por `uid` y, en el último bloque, la regla exige lo mismo.

src/stores/tareas.ts

```
import { computed, ref } from 'vue'
import { defineStore } from 'pinia'
import { addDoc, collection, deleteDoc, doc, query, updateDoc, where } from
   'firebase/firestore'
import { useCollection } from 'vuefire'
import { db } from '../services/firebase'
import { useSesionStore } from './sesion'
import type { Estado, Filtro, Tarea } from '../types'

const SIGUIENTE: Record<Estado, Estado> = {
  pendiente: 'en-curso',
  'en-curso': 'hecha',
  hecha: 'hecha',
}

export const useTareasStore = defineStore('tareas', () => {
  const sesion = useSesionStore()
  const filtro = ref<Filtro>('todas')

  // Una consulta viva: si cambia el usuario, VueFire cancela la anterior y abre otra
  const {
    data,
```

```
    pending: cargando,
    error,
  } = useCollection<Tarea>(() =>
    sesion.usuario ? query(collection(db, 'tareas'), where('uid', '==', sesion.usuario.id))
   : null,
  )
  const tareas = computed(() => [...data.value].sort((a, b) => a.creada - b.creada))

  const visibles = computed(() =>
    filtro.value === 'todas' ? tareas.value : tareas.value.filter((t) => t.estado ===
   filtro.value),
  )
  const pendientes = computed(() => tareas.value.filter((t) => t.estado !== 'hecha').length)

  // Escribimos en Firestore; la lista se actualiza sola cuando la consulta avisa
  const agregar = (titulo: string, proyectoId = 1) =>
    addDoc(collection(db, 'tareas'), {
      titulo,
      proyectoId,
      estado: 'pendiente',
      uid: sesion.usuario?.id,
      creada: Date.now(),
    })
  function avanzar(id: string) {
    const tarea = tareas.value.find((t) => t.id === id)
    if (tarea) updateDoc(doc(db, 'tareas', id), { estado: SIGUIENTE[tarea.estado] })
  }
  const eliminar = (id: string) => deleteDoc(doc(db, 'tareas', id))

  return { tareas, cargando, error, filtro, visibles, pendientes, agregar, avanzar, eliminar
   }
})
```

Protegerlo y que funcione sin red

CONCEPTO localCache

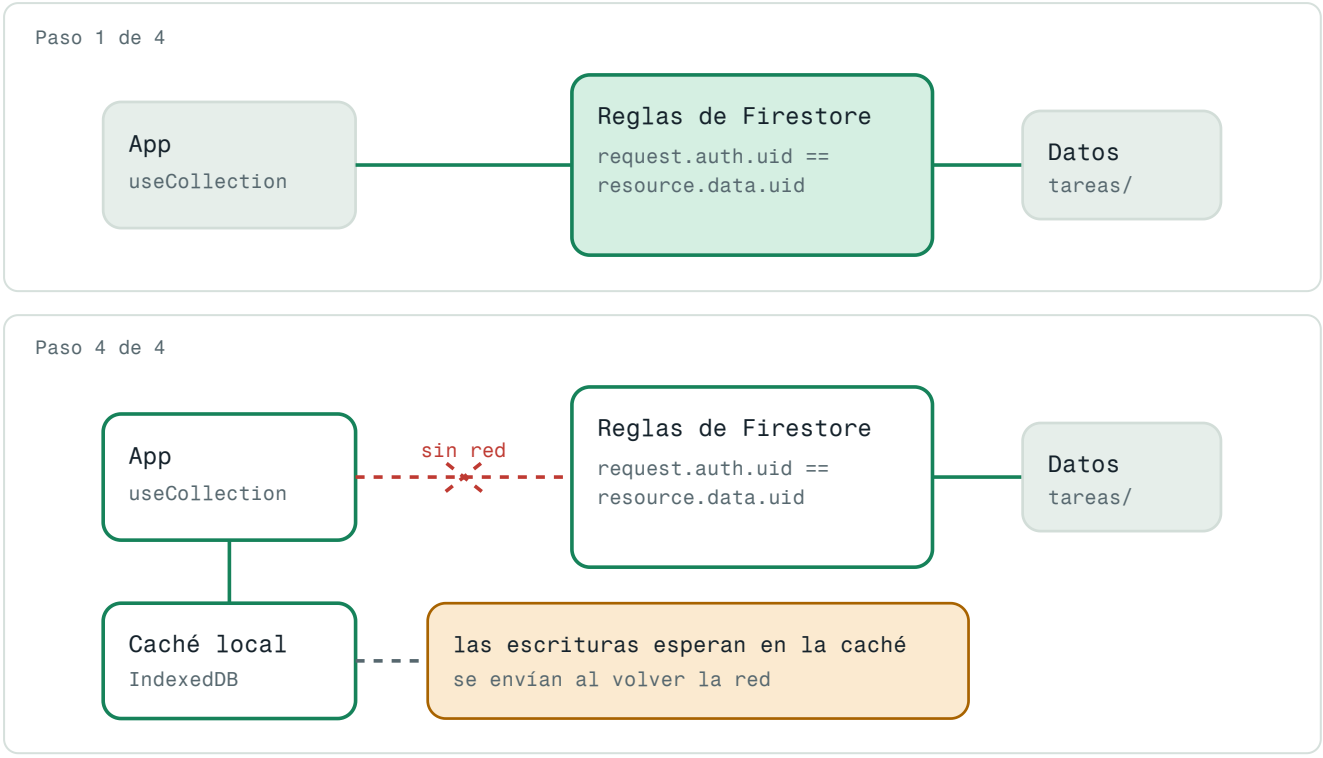
Las tareas ya viven en la nube. ¿Qué impide que otro usuario las lea, y qué pasa si se cae la red?

QUÉ ES

Las reglas de Firestore deciden en el servidor quién lee y escribe; la caché persistente guarda los datos en el dispositivo.

PARA QUÉ SE USA

Datos protegidos por usuario y una app que sigue funcionando sin conexión.



Cómo funciona, paso a paso

1. Reglas por usuario
2. Publicar y probar
3. Caché persistente
4. Sin conexión

REGLA

Ocultar un botón **no protege** nada: la regla se escribe y se prueba en el servidor.

LO QUE AGREGAMOS A TASKFLOW

- reglas: cada quien ve lo suyo
- app que funciona sin red

Bloque R4 de 4 · Reglas y modo sin conexión

1. **Consola** → **Firestore** → **Reglas** — reemplaza las de prueba por las de abajo y pulsa **Publicar**. Están en `src/firestore.rules`.
2. **Navegador** — crea otra cuenta: no ve las tareas de la primera. Si quitas el `where` de la consulta, Firestore responde `permission-denied`: **rechaza** las consultas que podrían traer documentos ajenos.
3. **services/firebase.ts** — cambia `getFirestore` por `initializeFirestore` con `localCache: persistentLocalCache(…)`: los datos quedan también en el dispositivo.
4. **Navegador** — en DevTools → *Network* → *Offline* aparece «Sin conexión»; lo que agregues se sincroniza al volver la red.

OJO

Ocultar un botón **no protege** nada: las reglas se escriben y se prueban en el servidor. Y las reglas de «modo de prueba» caducan: no las dejes.

src/firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /tareas/{tareaId} {
      allow read, delete: if resource.data.uid == request.auth.uid;
      allow create: if request.resource.data.uid == request.auth.uid;
      allow update: if resource.data.uid == request.auth.uid
                    && request.resource.data.uid == request.auth.uid;
    }
  }
}
```

src/services/firebase.ts

```ts
import { initializeApp } from 'firebase/app'
import {
  initializeFirestore,
  persistentLocalCache,
  persistentMultipleTabManager,
} from 'firebase/firestore'

// Estas claves no son secretas: solo identifican tu proyecto. La seguridad la dan las reglas.
export const firebaseApp = initializeApp({
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
})

// La caché en el dispositivo (IndexedDB) deja leer y escribir sin red; al volver, se sincroniza
export const db = initializeFirestore(firebaseApp, {
  localCache: persistentLocalCache({ tabManager: persistentMultipleTabManager() }),
})
```

CIERRE · PARA SEGUIR

Cuatro preguntas, cuatro respuestas

PREGUNTA	RESPUESTA
¿Cómo conectamos la app a un backend real?	`initializeApp` + plugin `VueFire`
¿Cómo sabe la app quién es el usuario?	`useCurrentUser` · `getCurrentUser()` en la guardia
¿Cómo mostramos las tareas sin recargar?	`useCollection` + `where('uid', …)`
¿Cómo protegemos los datos y trabajamos sin red?	reglas de Firestore · `localCache`

Para practicar

Agrega a TaskFlow el botón «Eliminar» que borra la tarea en Firestore con `deleteDoc` y comprueba en una segunda pestaña que desaparece sola.

Glosario

TÉRMINO	EN UNA FRASE
SDK modular	Librería de Firebase en la que importas solo las funciones que usas.
VueFire	Plugin que conecta Firebase con Vue y entrega refs reactivos.
Firebase Auth	Servicio de cuentas: correo, proveedores externos como Google y sesión.
MFA	Segundo factor (SMS o app) además de la contraseña.
Colección · documento	Firestore guarda documentos (objetos) dentro de colecciones.
Consulta · `onSnapshot`	Pide documentos filtrados; Firestore avisa en cada cambio.
Reglas de seguridad	Código que corre en el servidor y decide quién lee y escribe cada documento.
Caché persistente	Copia de los datos en el dispositivo (IndexedDB) para trabajar sin red.

Para repasar

VueFire · documentación	`vuefire.vuejs.org`
Firebase Auth en la web	`firebase.google.com`
Reglas de seguridad de Firestore	`firebase.google.com`
Datos sin conexión	`firebase.google.com`